EPFL ETP

River Flow Mini-Project Report

Redouan El Âaddaoui, Jean Lemercier, Thomas Poupard

November 13, 2026

1 Meshing

Meshing strategies

Create the *Named Selections* for the boundary

- Rhine_in for the inlet of the Rhine
- Neckar_in for the inlet of the Neckar
- Rhine_out for the outlet of the Rhine
- Wall_1, Wall_2, Wall_3 for the three riverbanks

With the mesh selected in the project tree, set *Physics Preference* to CFD and *Element Size* to 15m under **Details** → **Defaults**. Add an *Inflation* and a *Method* property to the mesh, as well as an *Edge Sizing* for all 6 edges (3 walls, 2 inlets, 1 outlet). Under **Details** → **Definition**, after having clicked on inflation in the project tree, set *Boundary Scoping Method* to Named Selections and *Boundary* to Wall_1 & Wall_2 & Wall_3 by holding down ctrl. We also increase *Maximum Layers* to 4 to get a finer mesh near the borders without altering the inner regions of the mesh too much. For each edge sizing, under **Details** → **Definition**, set *Type* to Factor of Global Size.

The meshing parameters to act upon are the *Method*, the inflation *Growth Rate*, and the edge sizing *Element Size Factor* for each boundary line. To determine a satisfactory combination of parameters, we use the Taguchi method. In order to keep the number of required experiments reasonably low, we use one element size factor per boundary type (walls vs. in/outlets). We differentiate them due to the significant difference in scale. We use the Taguchi L9 orthogonal array, which is suitable for four 3-level factors.

	1	2	3	4
1	1	1	1	1
2	1	2	2	2
3	1	3	3	3
4	2	1	2	3
5	2	2	3	1
6	2	3	1	2
7	3	1	3	2
8	3	2	1	3
9	3	3	2	1

Table 1: Taguchi L9

The values to be tested for each parameter are chosen to be realistic and reasonable while being different enough to have a noticeable impact on the resulting mesh, in order to have a meaningful evaluation and comparison. Ultimately, this somewhat comes down to trial-and-error and empirical judgment.

Parameter	Levels		
	1	2	3
method	Quadrilateral Dominant	Triangles	MultiZone Quad/Tri
inflation growth rate	1.2	1.4	1.6
walls element size factor	0.5	0.75	1.0
in/outlets element size factor	0.5	0.75	1.0

Table 2: Meshing parameters design levels

The desired mesh quality metric can be set under **Details** → **Quality** → *Mesh Metric* after having selected the mesh in the project tree. The minimum, maximum, average and standard deviation of the metric can be viewed there as well. To avoid complications with multi-objective optimization, we solely maximize the average value, while simply monitoring the min and std. More information about the mesh quality metric can be found in section 1. The algorithm to determine the optimal parameters combination is implemented in the file *taguchi.py*

Experiment	Mesh quality metric		
	mean	std	min
1	0.98564	3.2559e-2	0.61695
2	0.99368	2.6767e-2	0.57822
3	0.99283	3.2757e-2	0.45389
4	0.97398	4.0407e-2	0.31426
5	0.97323	4.0123e-2	0.21215
6	0.97283	3.9929e-2	0.51198
7	0.99356	2.4473e-2	0.64168
8	0.98953	2.0809e-2	0.8283
9	0.99132	2.3951e-2	0.72091

Table 3: Taguchi design experiments results

Meshing method	**Inflation growth**	**Wall size factor**	**Inlet/outlet size factor**
MultiZone Quad/Tri	1.6	1.00	0.75

Table 4: Optimized meshing parameters

Another parameter we could have acted upon is the *Bias Type* on all edge sizing properties. We tried using the — – - – — bias type on the inner wall between the Rhine and the Neckar, with the intention of increasing the mesh density near the junction, which is the most critical zone in the system. This however created distortions in the mesh, that arise because cells across the river from one another are forced to have different sizes. This effect also prevents us from introducing a bias on the other river banks, since the lines are not of the same length, and therefore cannot match. A solution to this problem would be to divide the walls into shorter

line, granting finer control on the biases and element sizes to try to achieve a more uniform meshing. Biases on the ports boundaries did not do any good either.

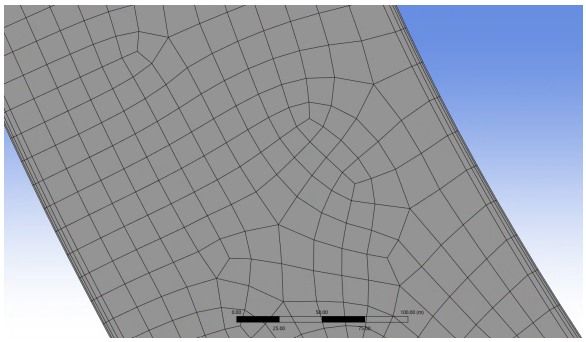


Figure 1: Mesh snapshot with biased inner wall ($Bias\ Factor = 5$)

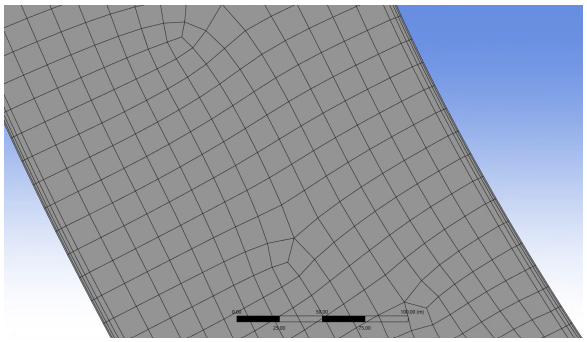


Figure 2: Mesh snapshot with no bias

Generated mesh

The optimization procedure settled for the MultiZone Quad/Tri method. It is the most flexible method, allowing the meshing algorithm to use both quadrilaterals and triangles. In our case, the mesh is composed solely of quadrilaterals (with most being close to squares), which coincides with our orthogonality requirements. The ports element size factor is less than that of walls, apriori to accommodate for the smaller spatial scale. The inflation growth rate is the highest among the possible options, resulting in a smoother transition between the tiny border cells and the larger inner cells.

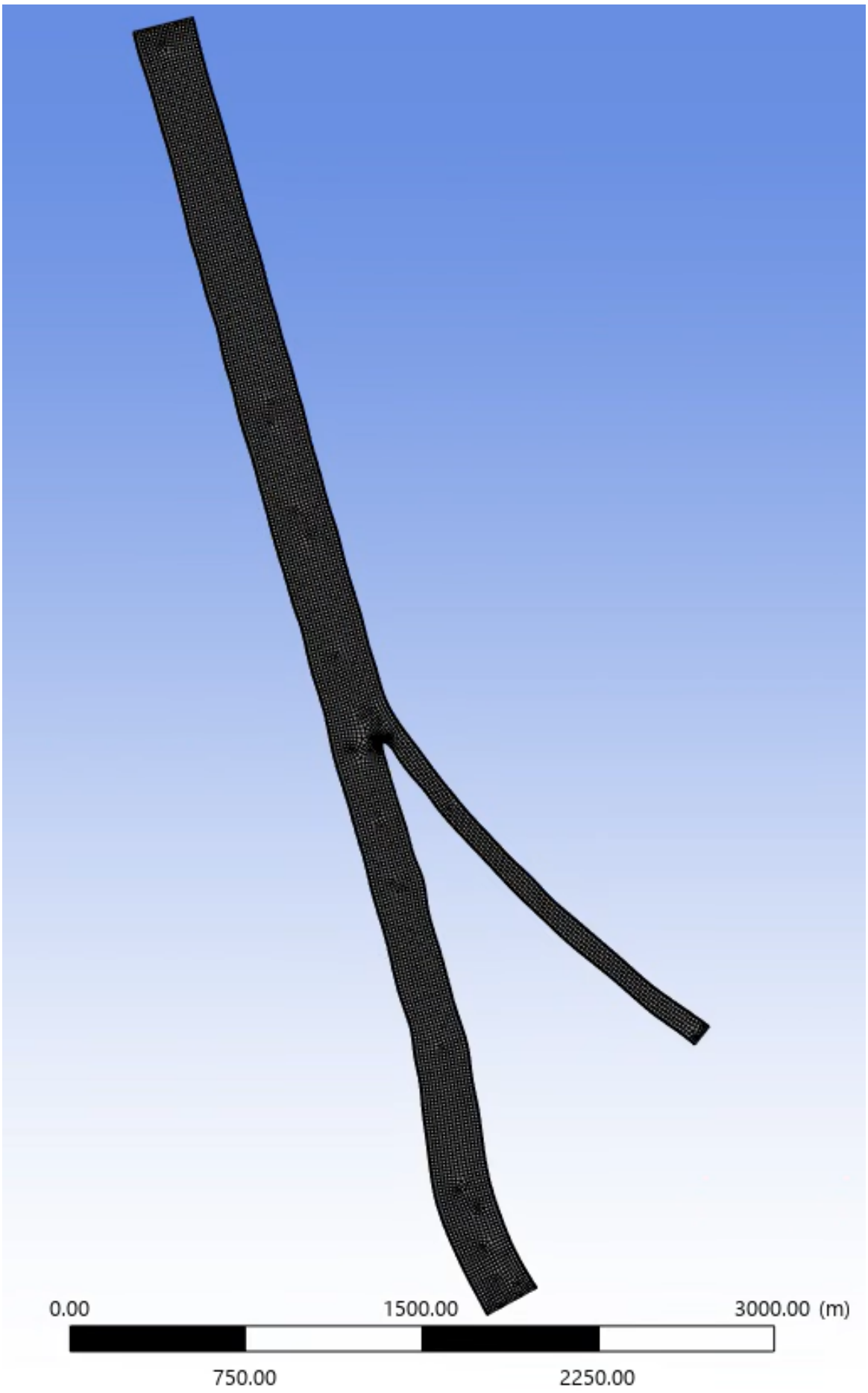


Figure 3: Visual representation of the generated mesh (global view)

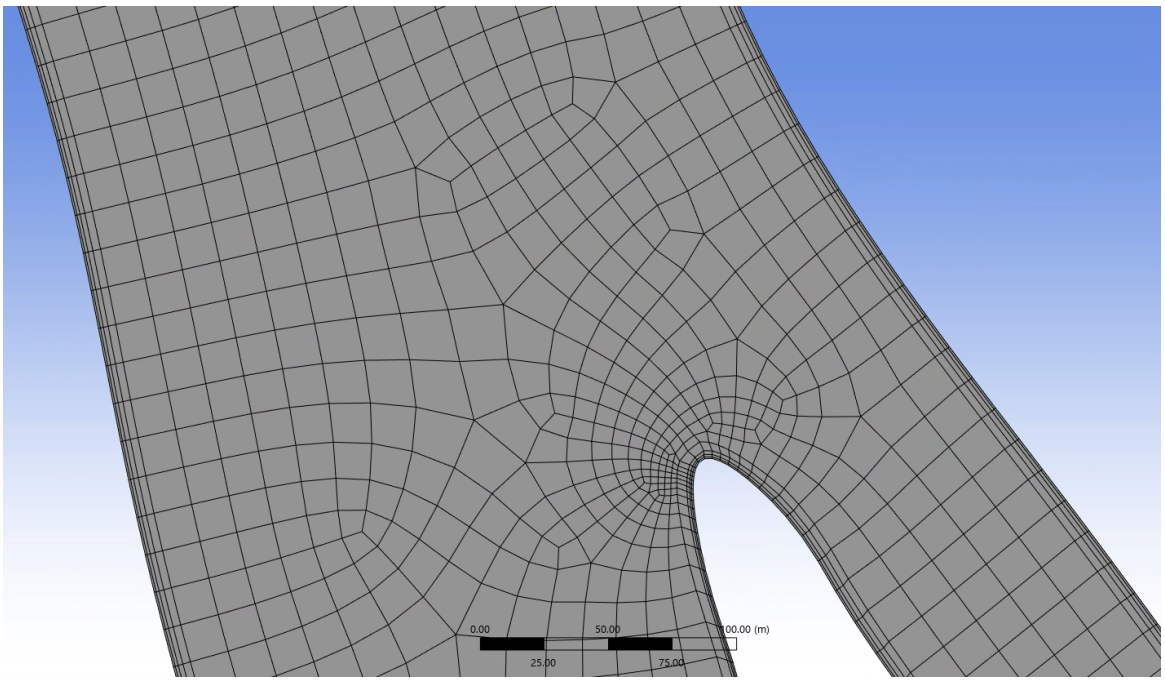


Figure 4: Visual representation of the generated mesh (junction)

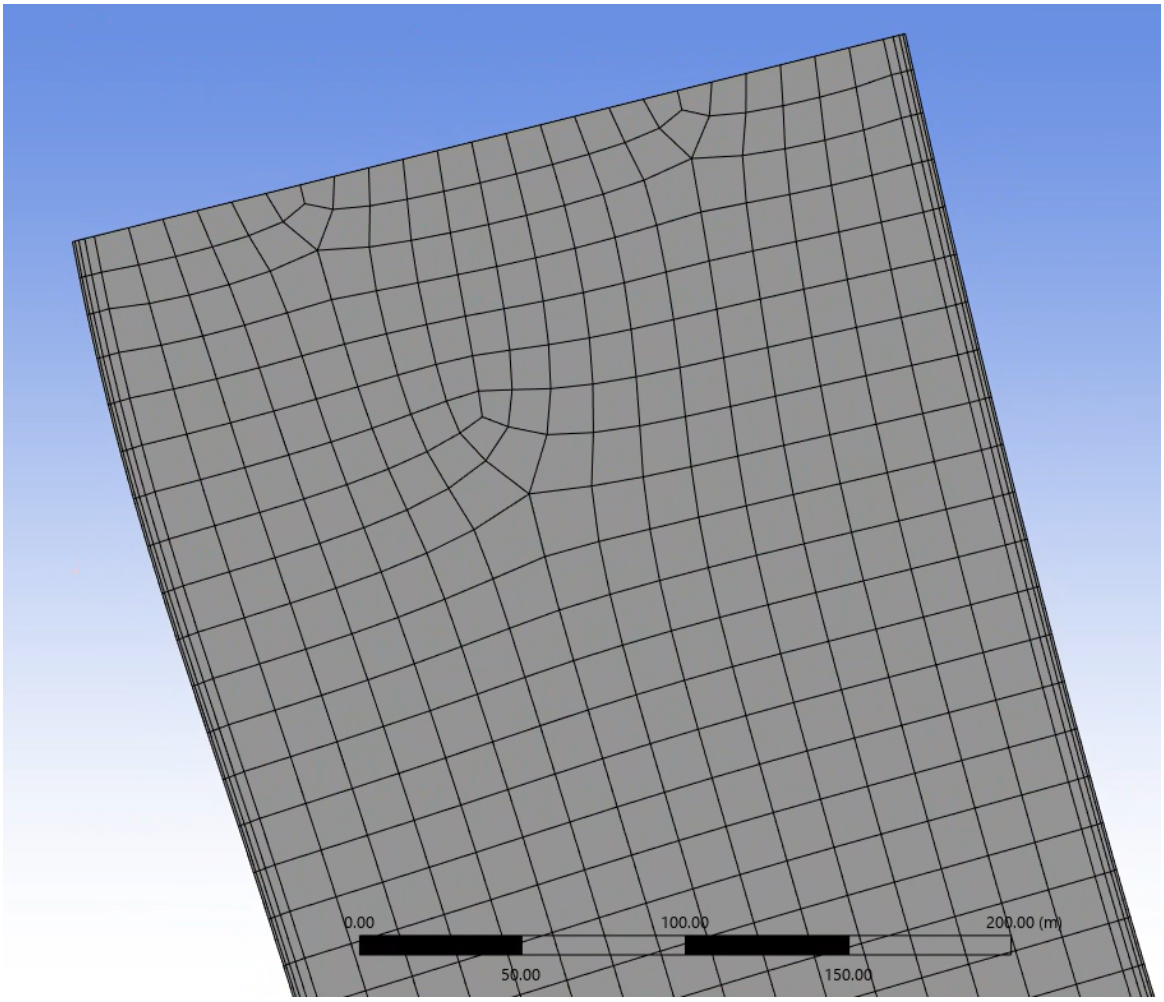


Figure 5: Visual representation of the generated mesh (outlet)

Mesh quality assessment

The mesh quality metric we chose is Orthogonal Quality, which ranges from 0 (worst) to 1 (best) for every cell. The official Ansys help page on mesh requirements and consideration cites it as one of the most important metric in Fluent. Orthogonality ensures better estimations of gradients and fluxes across cells, which is especially relevant for our confluent rivers flow modeling, where gradients near borders and the junction vary rapidly. The help page states that "the minimum orthogonal quality for all types of cells should be more than 0.01, with an average value that is significantly higher."

Min	**Max**	**Mean**	**Std**
0.7277	1.0000	0.99485	2.1782×10^{-2}

Table 5: Mesh quality assessment results through orthogonal quality

The average and minimum values are higher than any value encountered in Taguchi design experiments, meaning that single-objective optimization of the mean was an acceptable choice. Critically, we note that the minimum value is vastly satisfactory relative to the recommendations.

2 Flow simulation

The simulation was set up with the following conditions:

- **General settings**
  - Solver type: Pressure-Based
  - Time: Steady
- **Models**
  - Turbulence model: Realizable k-ε with Standard Wall Functions
  - Energy equation: ON
- **Materials**
  - Fluid: water-liquid
- **Boundary conditions**
  - Neckar_in: velocity magnitude $U = 4$ m/s, temperature $T = 300$ K
  - Rhine_in: velocity magnitude $U = 2$ m/s, temperature $T = 290$ K
  - Rhine_out: Gauge Pressure 0 Pa
- **Solution methods**
  - Pseudo Time Methods : OFF
- **Initialization and calculation**
  - Initialization: standard, computed from `Rhine_in`
  - Convergence criteria: residuals below 10^{-6} for all equations except continuity
  - Maximum number of iterations: 1000

Realizable k-ε turbulence model

The mean water levels at the gauging stations located at the confluence in Mannheim are approximately $h_{\text{Rhine}} \approx 2.93$ m[*] for the Rhine and $h_{\text{Neckar}} \approx 2.81$ m[**] for the Neckar.

These values are used as an approximation of the flow depth and therefore as the characteristic length L of each river.

The Reynolds number is defined as:

$$Re = \frac{\rho U L}{\mu} \tag{1}$$

where $\rho \approx 1000$ kg/m^3 and $\mu \approx 10^{-3}$ Pa·s are the density and dynamic viscosity of water, and $U = 4$ m/s is the inlet velocity.

[*] `https://www.agrarwetter-aktuell.de/flusspegel/rhein-mannheim-station.23700700.html`

[**] `https://www.wetteronline.de/?gid=BWB&iid=Neckar&pcid=pc_pegel_data&sid=River`

For the Rhine:

$$Re_{\text{Rhine}} = \frac{998.2 \times 2 \times 2.93}{1.003 \times 10^{-3}} \approx 6 \times 10^{6} \tag{2}$$

For the Neckar:

$$Re_{\text{Neckar}} = \frac{998.2 \times 4 \times 2.81}{1.003 \times 10^{-3}} \approx 1.12 \times 10^{7} \tag{3}$$

Both Reynolds numbers are several orders of magnitude above the critical value for transition, so the flow is turbulent. This justifies the use of a turbulence model.

The realizable k-ε turbulence model is built from several modelling steps that allow turbulent flows to be simulated with good accuracy, except in the near-wall region, where wall functions are required:

- Reynolds decomposition of the velocity and pressure into a mean and a fluctuating part, leading to the RANS equations:

$$u_i = U_i + u_i', \qquad p = P + p' \tag{4}$$

$$\rho\left(\frac{\partial U_i}{\partial t} + U_j\frac{\partial U_i}{\partial x_j}\right) = -\frac{\partial P}{\partial x_i} + \frac{\partial}{\partial x_j}\left[\mu\frac{\partial U_i}{\partial x_j} - \rho\,\overline{u_i'u_j'}\right] \tag{5}$$

- Boussinesq hypothesis, which models the Reynolds stresses through a turbulent viscosity μ_t:

$$-\rho\,\overline{u_i'u_j'} = \mu_t\left(\frac{\partial U_i}{\partial x_j} + \frac{\partial U_j}{\partial x_i}\right) - \frac{2}{3}\rho k\,\delta_{ij}, \qquad k = \frac{1}{2}\overline{u_i'u_i'} \tag{6}$$

- Dimensional analysis to express the turbulent viscosity, with a variable coefficient C_μ that depends on the mean strain and rotation rates, ensuring that the normal stresses remain positive (realizability):

$$\mu_t = \rho\,C_\mu\frac{k^2}{\varepsilon}, \qquad C_\mu = \frac{1}{A_0 + A_s\dfrac{k\,U^*}{\varepsilon}} \tag{7}$$

  where $U^* = \sqrt{S_{ij}S_{ij} + \tilde{\Omega}_{ij}\tilde{\Omega}_{ij}}$, $A_0 = 4.04$ and $A_s = \sqrt{6}\cos\phi$, with $\phi = \frac{1}{3}\arccos\left(\sqrt{6}\,W\right)$, $W = \dfrac{S_{ij}S_{jk}S_{ki}}{\tilde{S}^3}$ and $\tilde{S} = \sqrt{S_{ij}S_{ij}}$.

- Transport equation for the turbulent kinetic energy k, closed with a gradient-diffusion hypothesis):

$$\frac{\partial(\rho k)}{\partial t} + \frac{\partial(\rho k U_j)}{\partial x_j} = \frac{\partial}{\partial x_j}\left[\left(\mu + \frac{\mu_t}{\sigma_k}\right)\frac{\partial k}{\partial x_j}\right] + P_k - \rho\varepsilon, \qquad P_k = \mu_t S^2 \tag{8}$$

  where $S = \sqrt{2S_{ij}S_{ij}}$ and $S_{ij} = \frac{1}{2}\left(\frac{\partial U_i}{\partial x_j} + \frac{\partial U_j}{\partial x_i}\right)$.

- Transport equation for the dissipation rate ε, derived from the exact transport equation of the mean-square vorticity fluctuation:

$$\frac{\partial(\rho\varepsilon)}{\partial t} + \frac{\partial(\rho\varepsilon U_j)}{\partial x_j} = \frac{\partial}{\partial x_j}\left[\left(\mu + \frac{\mu_t}{\sigma_\varepsilon}\right)\frac{\partial \varepsilon}{\partial x_j}\right] + \rho C_1 S\varepsilon - \rho C_2\frac{\varepsilon^2}{k + \sqrt{\nu\varepsilon}} \tag{9}$$

  C_2, σ_k and σ_ε are empirical constants calibrated on reference flows, whereas C_1 and C_μ are not constant and adapt to the local mean strain and rotation rates.

As mentioned above, the near-wall region requires a special treatment. The k-ε model is a high Reynolds number model and is therefore not valid in the viscous sublayer, where molecular viscosity dominates. Moreover, the Boussinesq hypothesis assumes an isotropic turbulent viscosity, whereas turbulence close to a wall is strongly anisotropic, since the wall damps the velocity fluctuations normal to it. For these reasons, wall functions are used to bridge the near-wall region.

Viscosity Ratio

By default, Fluent limits the turbulent viscosity ratio μ_t/μ to 10^5.When this limit is reached, μ_t is clipped.

The turbulent viscosity ratio scales as

$$\mu_t = \rho C_\mu \frac{k^2}{\varepsilon}, \quad k \sim u'^2, \quad \varepsilon \sim \frac{u'^3}{\ell} \quad \Rightarrow \quad \frac{\mu_t}{\mu} \sim \frac{\rho\, u'\, \ell}{\mu} \tag{10}$$

where u' is the velocity fluctuation and ℓ the size of the largest eddies.

For a large-scale river flow such as the Rhine, this limit can be exceeded physically. The largest eddies are of the order of the river depth, so ℓ is large, while the molecular viscosity of water is small.

Capping μ_t would artificially reduce the turbulent diffusion, giving a non-physical solution and slowing down convergence. Increasing the limit to 10^{10} therefore lets the turbulence model compute μ_t freely.

Results

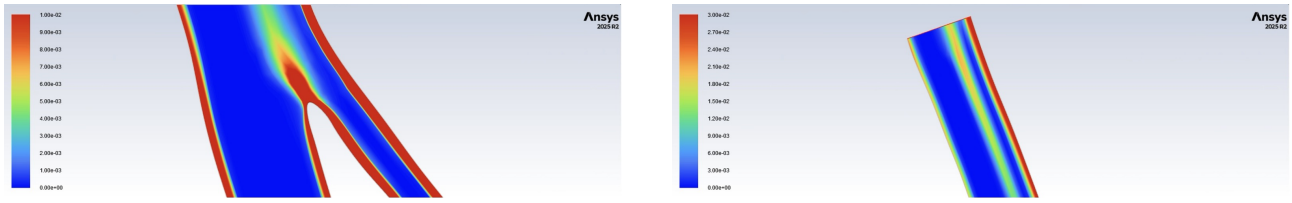
The simulation converges after 221 iterations, which is lower than our maximum number of iterations (1000).

The first feature we wanted to analyze is the turbulent kinetic energy k, defined by:

$$k = \frac{1}{2}\left(\overline{u'^2} + \overline{v'^2} + \overline{w'^2}\right) \tag{11}$$

where u', v' and w' are the velocity fluctuations in the three directions.

We can notice that the turbulent kinetic energy is higher near the walls and that a wake appears at the junction between the two rivers (Figure 6a). Near the walls, the strong velocity gradients due to the no-slip condition increase the production of turbulence. At the junction, the two flows meet with different velocities and directions: the Neckar enters with a velocity of 4 m/s, twice that of the Rhine (2 m/s). This creates a shear layer and a wake downstream of the confluence corner, where k reaches its maximum values. Moreover, downstream of the junction, the velocity gradients decrease, so the production of turbulence drops while dissipation remains active. As a result, the turbulent kinetic energy progressively decays along the wake, which also widens as the two flows mix.
However, since the transverse mixing between the two rivers is weak, this turbulent region remains on the right side of the river, where the Neckar enters. At the outlet section (Figure 6b), the turbulent kinetic energy is therefore higher on the right side than on the left side.

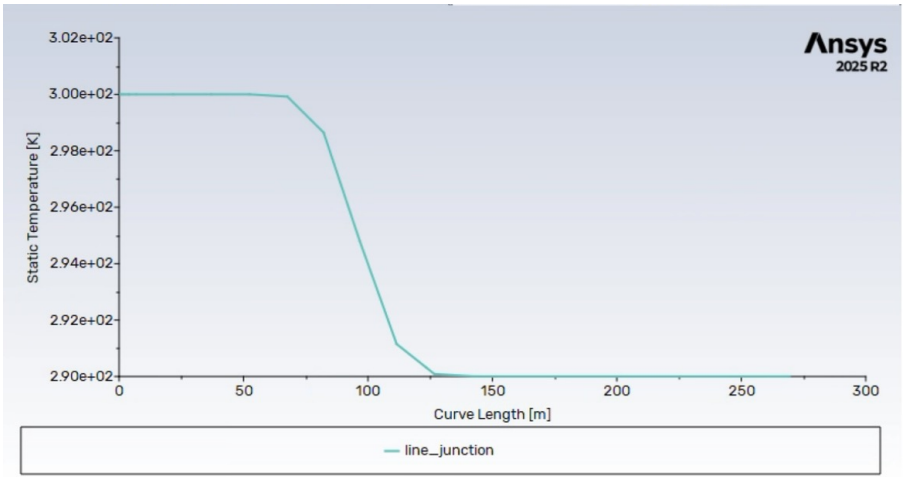


(a) TKE at the junction. (b) TKE at the outlet

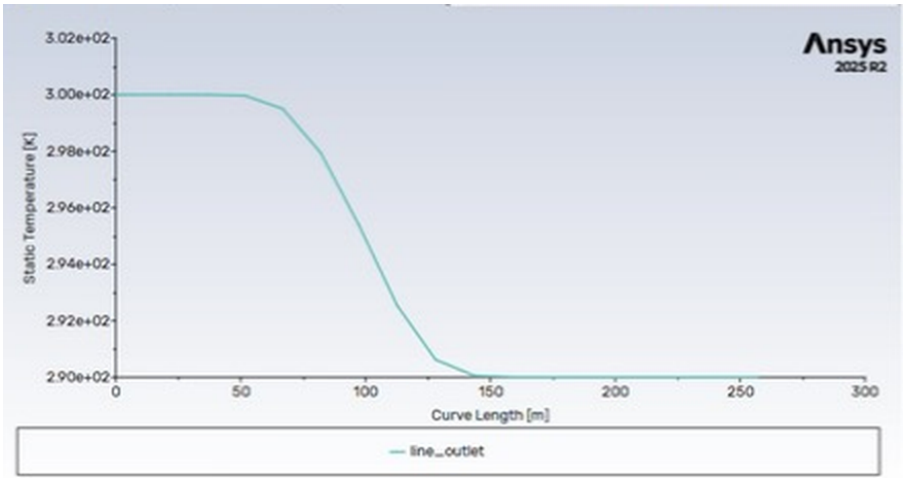
Figure 6: Zoomed views of the turbulent kinetic energy k contour.

Secondly, we analyzed the temperature distributions at two different sections,one just after the junction and one just before the outlet (Figure 7). The two distributions are almost identical: a zone at 300 K between 0 and 60 m, a zone at 290 K between 150 and 300 m, and a region with a strong temperature gradient in between.

The quality of the temperature mixing is therefore quite low: the two flows remain thermally separated over the whole simulated length, and only a narrow transition region develops between them. However, the temperature profile before the outlet is smoother than after the junction, with less abrupt changes in slope, which indicates that the mixing has started to progress downstream.



(a) Temperature distribution just after the junction.



(b) Temperature distribution just before the outlet.

Figure 7: Temperature distributions at two sections of the domain.

Finally, we computed the area-weighted average temperature at `Rhine_out`:

$$\bar{T}_{\texttt{Rhine_out}} = \frac{1}{A}\int_A T\,dA = 293.87305 \text{ K} \tag{12}$$

Area-Weighted Average Static Temperature	[K]
rhine_out	293.87305

Figure 8: Fluent console output of the area-weighted average temperature at `Rhine_out`.